边界值点积测试说明

本测试集共包含 45 例边界值点积测试向量，目前已完整合入 939 例综合测试程序中，并在香山昆明湖 V2 处理器与 NEMU 模拟器协同仿真环境下全部完成 128 位整寄存器比对、顺利到达预设成功地址。本文档重点说明该类边界用例的极值覆盖设计、预期真值推导及在综合回归中的对应区间；相关的环境准备、构建步骤与结果判定准则统一参见项目共用手册。

1 测试范围与输入

本专项测试旨在全面考核硬件点积单元在输入元素极值（−128、127）、全零输入、累加器输出极值、固定 8 元素窗口首尾边界以及高 8 元素屏蔽隔离等极端条件下的硬件行为。测试集由 15 组人工定向构造的边界用例与 30 组极值随机组合用例共同构成。

本组测试激励文件位于 tests/cases_bound.txt，配套用例标签说明位于 tests/cases_bound.labels.csv，便于逐字节核对的合并表格位于 tests/cases_readable.csv。下表列出了前 15 组定向构造边界用例的详细参数。编号从 0 起算，元素按低字节至高字节顺序排列，所有用例的目标寄存器高 96 位预期输出均为全零：

序号	两个源寄存器输入	期望点积值	目标低 32 位编码
0	A 全为 -128 B 全为 -128	131072	0x00020000
1	A 全为 -128 B 全为 127	-130048	0xfffe0400
2	A 全为 0 B 全为 0	0	0x00000000
3	A 全为 0 B 全为 127	0	0x00000000
4	A -128, 0, 0, 0, 0, 0, 0, 0 B -128, 0, 0, 0, 0, 0, 0, 0	16384	0x00004000
5	A 0, 0, 0, 0, 0, 0, 0, -128 B 0, 0, 0, 0, 0, 0, 0, -128	16384	0x00004000
6	A 全为 0 B 全为 0 高八元素 A -128, 0, 0, 0, 0, 0, 0, 0；B -128, 0, 0, 0, 0, 0, 0, 0	0	0x00000000
7	A 全为 0 B 全为 0 高八元素 A 全为 -128；B 全为 -128	0	0x00000000
8	A 全为 -128 B 全为 -128 高八元素 A 全为 -128；B 全为 -128	131072	0x00020000
9	A 全为 127 B 全为 -128	-130048	0xfffe0400

序号	两个源寄存器输入	期望点积值	目标低 32 位编码
10	A -128, 127, -128, 127, -128, 127, -128, 127 B 全为 1	-4	0xfffffffc
11	A -128, -128, -128, -128, -128, -128, -128, 127 B 全为 -128	98432	0x00018080
12	A -128, 0, 0, 0, 0, 0, 0, 127 B -128, 0, 0, 0, 0, 0, 0, 127	32513	0x00007f01
13	A 1, 2, 3, 4, 5, 6, 7, 8 B 8, 7, 6, 5, 4, 3, 2, 1	120	0x00000078
14	A 100, 0, 0, 0, 0, 0, 0, 0 B 100, 0, 0, 0, 0, 0, 0, 0	10000	0x00002710

2 预期结果与检查重点

从累加位宽极值来看，当 8 对输入全部取 −128 时，点积达到理论正向最大值 8×(−128)×(−128)=131072；当两个源分别全取 −128 与 127 时，点积达到理论负向最小值 8×(−128)×127=−130048。由于正向极值 131072 恰好超出 18 位有符号补码的表示上限 131071 达 1 个单位，硬件加法树累加通路必须保留至少 19 位有符号位宽方可避免溢出。

在窗口边界校验上，第 4 号与第 5 号用例分别单独激励固定窗口的首元素与末元素；第 6 号与第 7 号用例将低 8 元素置零、同时向高 8 元素注入 −128 极值，用于确认高半区数据绝不混入点积求和；第 8 号用例进一步将整个 128 位寄存器的全部 16 个元素均设为 −128，期望输出仍精确保持为 8 路极值之和 131072。此外，第 10 号用例通过交替排布四项 −128 与四项 127 并与全 1 向量作点积，验证正负极值交错抵消下的细粒度算术精度（4×(−128)+4×127=−4）。

3 完整处理器结果对应

在全芯片回归测试中，本组 45 例边界值向量对应综合测试程序 programs/corpus_vl8_ta1.S 的第 95 至第 139 号测试项，该综合程序依次串联了 46 例正数、48 例负数、45 例边界值及 800 例随机向量。测试程序在每次执行点积前均先向目标寄存器 v4 写入非零背景数据，随后调用 vdot.vv 并校验完整的 128 位目标寄存器结果；任一用例出现失配均会立即跳转至失败地址。

随包归档的完整处理器协同仿真记录位于 results/cluster/20261019/corpus_vl8_ta1_r0.log，全量回归摘要记录于同目录下的 summary.json。实测结果表明，该综合测试程序在开启 NEMU 同步比对下顺利执行完毕，最终停机地址为 0x80000088，与程序规定的成功退出地址完全一致。

本类 45 例边界值测试已随 939 例综合程序全部验证通过。若需单独拉起本分类测试程序开展专项复核，可按下一节给出的指令执行，其底层输入向量与预期结果与前文完全一致。

4 单独运行与结果判定

完整处理器的编译流程统一参见《2.3.0_共用环境部署与操作手册》第 6 章完整处理器构建与协同验证。若当前工作区尚未构建香山仿真器与 NEMU 参考库，请先按该章节完成构建。

分类测试的执行规范统一参见《2.3.0_共用环境部署与操作手册》第 5 章基础功能与测试程序。单独拉起本组边界值测试时需指定参数 --group bound；若需运行包含本组的全量综合程序，则指定 --group corpus：

```
python3 -B scripts/prepare_tests.py --out "$RUN_ROOT/programs"
python3 -B scripts/run_tests.py --run-root "$RUN_ROOT" --group bound
```

在已按共用手册第 6.1 节完成处理器构建的 Linux 终端内，保持 RUN_ROOT 变量与工具链环境有效并执行上述命令。测试通过时，终端将输出 group=bound cases=45、预设成功终止地址及 BASIC_TEST_PASS 标志。单程序运行记录保存为 RUN_ROOT/basic_results/basic_bound.txt，汇总表保存为 RUN_ROOT/basic_results/latest_results.csv。

若需运行包含本组用例的 939 例综合测试程序，请执行：

```
python3 -B scripts/run_tests.py --run-root "$RUN_ROOT" --group corpus
```

本地独立算术模块的核验方法统一参见《2.3.0_共用环境部署与操作手册》第 3 章本地配套程序检查。若仅需在模块级检查两套 RTL 电路的算术正确性，可直接调用该章节的基础功能检查脚本；模块级仿真结果与完整处理器回归结果分目录独立存放。

测试输出路径与常见异常处理统一参见《2.3.0_共用环境部署与操作手册》第 8 章结果位置与使用说明。重新运行本分类测试所生成的记录文件固定保存在当前工作区的 basic_results/basic_bound.txt。

verification/check.py 另提供 48 组定向输入，对八个有效字节位置逐一检查，并覆盖高八字节不参与计算、正负极值、结果高 96 位清零和负结果扩展。两种随附电路各自与独立 C、Python 计算的完整 128 位结果比较。原 45 例边界输入及 939 例综合输入不改写。

检查程序实际生成 41 种错误实现：八种遗漏位置、28 种仅交换一个输入的位置，以及无符号乘法、18 位累加、误用高八位置、高位脏值和缺少符号扩展。每种错误至少必须被一个输入发现。

两个输入同时交换相同位置属于纯点积的等价变换。28 种同步交换应保持结果相同，不将其作为必须检测的错误。源目的寄存器重合及非法向量状态继续使用已有完整处理器专项程序核对。

操作入口见共用手册第 9.4 节；各错误类型与原有、新增检查的对应关系见 verification/coverage.csv。